PayFlow Database Portfolio

MIT 8103 Advanced Database Systems
Master of Information Technology | Miva University
2026/2027 First Semester Continuous Assessment

Odekunle Ayoola | Student ID: 301904401
Case study: fictional Nigerian digital wallet and transaction platform

Repository: github.com/cyberpastey/advanced-database-portfolio

1. Database design and modelling

PayFlow serves wallet customers, operations staff, fraud analysts and database administrators. PostgreSQL 17 stores customers, NGN wallets, immutable transfers, two signed ledger entries per movement and an event outbox. Opening funds are balanced against a system clearing wallet. Transfers, synthetic funding/withdrawals and compensating reversals use one database function; no external payment service is connected.

NUMERIC(18,2), primary/foreign keys, unique request keys, deferred posting checks and restricted application writes protect the model. Balances are derived from the ledger. Customer data is separated from wallets and transfer facts through 3NF; the outbox is an intentional event snapshot. The schema loaded 2,000 synthetic customers and 2,001 wallets. Nine negative tests rejected duplicate keys, invalid references/inputs, incomplete postings, history changes and direct application ledger writes.

2. Query processing and optimisation

Eight query result sets cover joins, aggregation, HAVING, subqueries, a CTE, window running balances, reversals and outbox audit. Two selective queries were benchmarked over 62,001 transfers using five measured EXPLAIN ANALYZE trials per condition after a warm-up. Results before and after indexing were equal.

Query	Rows	Before median	Indexed median
10-minute audit	600	15.223 ms	0.461 ms
High-value review	2,000	20.129 ms	1.590 ms

Plans changed from sequential scans to bitmap index/heap access; sorting remained. First measured shared-buffer hits fell from 1,387 to 14 and 31 respectively. These are warm-cache, single-host observations with baseline measured first, not production throughput guarantees.

Evidence: portfolio-1-database-design/evidence/verification.json; portfolio-2-query-optimisation/execution_plans/benchmarks.json and evidence/query_results.json.

3. Transactions and concurrency

A committed NGN 100 transfer debited wallet 2 and credited wallet 3 exactly. A forced division-by-zero after posting rolled back the complete operation: balances were unchanged and no transfer persisted. A compensating reversal retained the original history. An idempotent retry returned the original transfer ID; reuse with a different amount was rejected.

Two independent sessions each attempted NGN 74,977.50 from a wallet holding NGN 99,970.00. One committed and the other rejected insufficient funds (23514). A separate experiment observed a real transaction-ID lock wait, followed by commit after release. Wallets are locked in ascending ID order before available funds are checked. The application role cannot insert raw ledger rows.

READ COMMITTED observed a balance change from 99,970 to 99,971 after another session committed. REPEATABLE READ retained its initial 99,971 snapshot despite a later credit. A SERIALIZABLE read-modify-write conflict produced 40001, requiring a complete transaction retry. After a normal database restart, the successful transfer remained and all ledger invariants passed. This is not a power-loss or backup-restore test.

4. NoSQL and advanced data models

MongoDB 8 stores synthetic risk events with wallet/transfer references, time, risk score and nested rule/device metadata. Strict validation checks required fields and score range; unique event IDs support deduplication. Wallet/time and risk-score indexes support investigation. The test inserted 100 events, read filtered results, updated one, deleted one, aggregated by type and confirmed 99 remained. Invalid scores and duplicate IDs were rejected.

Flexible event documents suit varying metadata, but MongoDB does not enforce PostgreSQL foreign keys or make cross-store delivery atomic. The ledger remains authoritative. The PostgreSQL outbox is implemented; a retrying event consumer, dead-letter monitoring and production fraud rules are proposed extensions.

5. Distributed and cloud database exercise

Two local PostgreSQL servers demonstrated asynchronous logical replication of an isolated probe table. A primary marker appeared on the subscriber; while the subscription was disabled, a second committed marker remained absent. Re-enabling replication produced catch-up and matching rows. Observed polling times were 218 ms initially and 109 ms after re-enabling; they are not a replication service guarantee.

This local exercise does not replicate the entire wallet schema and does not provide physical standby, automatic failover or cloud availability. Logical replication does not automatically copy DDL or sequence state [3]. The proposed cloud design uses separate zones, fenced writer failover, TLS, least privilege and independently tested backups/PITR. Sharding by wallet complicates cross-wallet atomicity; time partitioning of events is a simpler early scaling option.

Evidence: P3 evidence/scenarios.json and durability.json; P4 evidence/operations.json; P5 evidence/replication.json. Each activity README links scripts, models, configuration and reproduction instructions. Full paths use the portfolio directories in the repository.

Technical reflection

The main design trade-off is to derive wallet balances from immutable ledger entries. This avoids a second mutable balance cache, while adding aggregation cost. Deferred integrity checks cannot replace concurrency control: ordered wallet locks and restricted write privileges are also necessary. Normalisation, atomicity and isolation address related but different failure modes.

A real bulk-load failure exposed the cost of holding 60,000 idempotency-key advisory locks in one transaction. Batching 500 movements per commit reduced lock lifetime while retaining atomicity for each transfer. Parameter escaping and explicit generate_series bound types also needed correction before the load passed. These events are documented in the decision/problem log.

The measured indexes improved both selected queries, but the plans still sorted and used heap access. The replication pause made eventual consistency visible: a committed primary row was temporarily absent downstream. MongoDB flexibility was useful for supplementary risk metadata, while financial truth remained relational. Production delivery retries, backup restoration and failover need further work.

References

[1] Miva University (2026). MIT 8103 Advanced Database Systems, 2026/2027 First Semester Continuous Assessment, pp. 1-8; supplied LMS instructions.
[2] PostgreSQL Global Development Group. Transaction Isolation; Explicit Locking; Using EXPLAIN. PostgreSQL 17 documentation: postgresql.org/docs/17/transaction-iso.html; explicit-locking.html; using-explain.html.
[3] PostgreSQL Global Development Group. Logical Replication: Restrictions. postgresql.org/docs/17/logical-replication-restrictions.html.
[4] MongoDB, Inc. Schema Validation; Unique Indexes and Schema Validation. mongodb.com/docs/manual/core/schema-validation/ and manual/tutorial/unique-indexes-schema-validation/.
[5] Psycopg 3 and PyMongo driver documentation. Full source and tool acknowledgements appear in documentation/references.md. Consulted 5 November 2026.

Submission notes

The implementation and results have been reviewed by the student. PDF is an accepted LMS format. The repository is publicly accessible at the student's request; the assessment specifies private visibility, so lecturer approval is needed for this arrangement. The original question document is unchanged, and no LMS submission has been made.